

pWin.ai Opportunities Quick Start

pWin.ai Opportunities helps you bootstrap a workspace from a company website, scan federal opportunities, turn promising items into capture briefs, and improve future matching with plain-English feedback.

Minimum Requirements

- One working host: OpenAI Codex , OpenClaw , or Claude Code
- python3 available in your shell
- A workspace folder where the skill can write artifacts
- A SAM.gov API key exposed as SAM_API_KEY
- An OpenAI API key exposed as OPENAI_API_KEY
- A reasoning-capable OpenAI model configured for your host
- o3-mini is fine for this setup
- Internet access
- GitHub access to this repo

This one-pager assumes an OpenAI-backed setup. If your host can talk to multiple model providers, use an OpenAI key for the steps below.

If your GitHub setup uses HTTPS instead of SSH, swap the clone URLs below to the HTTPS form you normally use.

1. Install the Skill

OpenAI Codex

```
git clone git@github.com:vlele/pwin-ai-opportunities-engine.git "$HOME/.codex/skills/pwin-ai-opportunities"
export PWIN_AI_OPPTS_ROOT="$HOME/.codex/skills/pwin-ai-opportunities"
```

If Codex does not pick it up immediately, restart Codex or refresh skill discovery.

OpenClaw

```
git clone git@github.com:vlele/pwin-ai-opportunities-engine.git "$HOME/.openclaw/skills/pwin-ai-opportunities"
export PWIN_AI_OPPTS_ROOT="$HOME/.openclaw/skills/pwin-ai-opportunities"
```

Claude Code

```
git clone git@github.com:vlele/pwin-ai-opportunities-engine.git "$HOME/src/pwin-ai-opportunities"
export PWIN_AI_OPPTS_ROOT="$HOME/src/pwin-ai-opportunities"
mkdir -p "$HOME/.claude/commands/pwin-ai-opportunities"
cp -R "$PWIN_AI_OPPTS_ROOT/claude-code/.claude/commands/" "$HOME/.claude/commands/pwin-ai-opportunities/"
```

Then copy or merge `claude-code/CLAUDE.md` into the Claude Code project context you normally use. After that, Claude Code users can call the command pack with prompts such as `/pwin-bootstrap`, `/pwin-scan`, `/pwin-research`, `/pwin-feedback`, and `/pwin-show-digest`.

2. Add Your Keys

In the shell where your host runs, export your keys:

```
export SAM_API_KEY="your-sam-gov-key"
export OPENAI_API_KEY="your-openai-key"
```

This quick start assumes `OPENAI_API_KEY` and a reasoning model. If you need a safe default model choice, `o3-mini` is fine.

The important part is that your host is configured to use a reasoning-capable OpenAI model rather than a non-reasoning default. `SAM_API_KEY` is the only skill-specific secret, but this guide assumes the LLM side is OpenAI.

3. Pick a Workspace

Create or open one folder per company:

```
mkdir -p "$HOME/work/acme-capture"
cd "$HOME/work/acme-capture"
```

This skill writes its outputs into `procurement/` inside that workspace.

4. Bootstrap the Workspace from the Company URL

In OpenClaw or Codex, ask:

```
Use pwin-ai-opportunities and bootstrap this workspace from https://example.com.
```

If you already know the NAICS codes, say:

Use `pwin-ai-opportunities` and bootstrap this workspace from `https://example.com` with confirmed NAICS 541511 and 541512.

If those NAICS are still tentative, say:

Use `pwin-ai-opportunities` and bootstrap this workspace from `https://example.com` and treat NAICS 541511 and 541512 as candidates.

In Claude Code, use the command pack:

```
/pwin-bootstrap workspace path $PWD and company URL https://example.com
/pwin-bootstrap workspace path $PWD and company URL https://example.com with confirmed
NAICS 541511 and 541512
/pwin-bootstrap workspace path $PWD and company URL https://example.com with candidate
NAICS 541511 and 541512
```

That creates or updates:

- `procurement/vendor-profile.json`
- `procurement/preferences.json`
- `procurement/source-registry.json`
- `procurement/STARTER_PROFILE.md`
- `MEMORY.md`

Bootstrap does more than create empty files. It immediately pre-populates `procurement/vendor-profile.json` with:

- bootstrap metadata and a `needs_user_confirmation` status
- `company.website` plus an inferred company name and summary
- inferred `core_competencies`, `other_taxonomy_tags.keywords`, and a starter `fit_narrative`
- confirmed `naics.confirmed` values when you provide confirmed codes
- candidate `naics.candidates` values from tentative input plus website inference
- candidate buyer notes, provenance facts, and a reminder that website-derived facts are still provisional

It also seeds `procurement/preferences.json` to exclude `grants` by default, prefer `contracts`, `subcontracts`, and `forecasts`, and reuse the inferred capability keywords. If `procurement/vendor-profile.json` already exists, bootstrap merges into it instead of resetting the file.

Then review `procurement/STARTER_PROFILE.md` and confirm or correct the provisional fields before the first real scan.

5. Run the First Scan

Once the workspace is bootstrapped, ask:

```
Use pwin-ai-opportunities and run a federal opportunity scan for the next 30 to 45 days in this workspace.
```

In Claude Code, use:

```
/pwin-scan workspace path $PWD and horizon 30-45
```

Expected first-run behavior:

- if `SAM_API_KEY` is missing, the scan reports `missing_api_key`
- if the workspace still has no usable NAICS, the scan reports `no_naics`
- if the scan finds matches, you will get a dated digest with stable IDs like `A1` , `W2` , or `E1`

6. Read the Digest

Ask:

```
Use pwin-ai-opportunities and show the latest digest.
```

In Claude Code, use:

```
/pwin-show-digest workspace path $PWD and date latest
```

7. Run Capture Research

Pick a stable ID from the digest and ask:

```
Use pwin-ai-opportunities and research A1 with full capture depth.
```

In Claude Code, use:

```
/pwin-research workspace path $PWD and entry A1 with full capture depth
```

That produces a fresh capture brief and evidence file under `procurement/capture-briefs/` and `procurement/capture-evidence/`.

8. Give Feedback

Use plain-English feedback such as:

```
never show grants  
like A1  
dislike W2 because too small  
more like A1
```

In Claude Code, the command-pack equivalent is:

```
/pwin-feedback workspace path $PWD and text never show grants  
/pwin-feedback workspace path $PWD and text like A1
```

That feedback is logged to `procurement/feedback-events.jsonl` and applied to future scans.

What Gets Created in Your Workspace

- `procurement/vendor-profile.json`
- `procurement/preferences.json`
- `procurement/source-registry.json`
- `procurement/STARTER_PROFILE.md`
- `procurement/digests/YYYY-MM-DD.md`
- `procurement/digest-entry-map/YYYY-MM-DD.json`
- `procurement/capture-briefs/ ...`
- `procurement/capture-evidence/ ...`
- `procurement/feedback-events.jsonl`
- `MEMORY.md`

Troubleshooting

- `missing_api_key` `SAM_API_KEY` is not exported in the shell running the scan.
- `no_naics` The workspace still does not have confirmed or candidate `NAICS` after bootstrap or manual edits.
- `HTTP 429` from `SAM.gov` Your key is wired correctly, but the API quota is throttled. Wait until the `nextAccessTime` returned by `SAM.gov` and rerun.

Short Version

1. Clone the repo with your normal authenticated GitHub method.
2. Export `SAM_API_KEY`.
3. Create a workspace.
4. Bootstrap it from the company website.
5. Review the starter profile and the populated `vendor-profile.json`.
6. Run the federal-only scan.
7. Read the digest, research a stable ID, and give feedback.

Direct Script Equivalents

These are helpful for debugging or CI. In normal OpenClaw and Codex usage, prefer the chat prompts above. In Claude Code, prefer the `/pwin-*` command pack shown above.

```
python3 "$PWIN_AI_OPPS_ROOT/scripts/bootstrap/bootstrap_workspace.py" --workspace "$PWD" --company-url "https://example.com"
python3 "$PWIN_AI_OPPS_ROOT/scripts/scan/run_scan.py" --workspace "$PWD" --horizon "30-45" --federal-only
python3 "$PWIN_AI_OPPS_ROOT/scripts/show/show_digest.py" --workspace "$PWD" --date latest
python3 "$PWIN_AI_OPPS_ROOT/scripts/capture/run_capture_research.py" --workspace "$PWD" --entry "A1" --depth full_360
python3 "$PWIN_AI_OPPS_ROOT/scripts/feedback/apply_feedback.py" --workspace "$PWD" --text "never show grants"
```